

Welcome to

# Self-hosting and Digital Sovereignty

## PROSA November 2026

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
self-hosting.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

# Contact information



- Henrik Kramselund, he/him internet samurai mostly networks and infosec
- Network and security consultant Zencurity, teacher and activist
- Master from the Computer Science Department at the University of Copenhagen (DIKU)
- Email: hlk@zencurity.com Mobile: +45 2026 6000
- Run a small network for fun AS57860

You are welcome to drop me an email

# Goals for today




**Take control of your own data and become independent of Big Tech**

Would you like to be more independent of major tech platforms like Google, Microsoft, and Amazon, but aren't sure where to start?

# Time schedule



Plan for the day

17:00 - 18:15 – Session 0: Introduction, the internet, and where to host

18:15 - 18:45 – Break

18:45 - 19:30 – Session 1: Technologies and securing your environment

19:30 - 19:45 – Break

19:45 - 20:30 – Session 2: Operations and running services in practice

20:30 - 21:00 – Discussion and questions



## Aarhus Kommune reducerer Microsoft-afhængighed med 25 procent over fire år

### Byrådet lagde kursen fast med klar procentsats

Aarhus Byråd traf i efteråret 2025 en konkret politisk beslutning: Over fire år skal kommunen reducere sin afhængighed af Microsofts kontorpakke med 25 procent. Beslutningen gav embedsværket et formelt mandat til at arbejde systematisk med digital handlefrihed og satte dermed gang i et bredere program, der rækker langt ud over kontorpakkens grænser.

Source:

https://www.lovguiden.dk/det-offentlige/staten-og-kommunernes-indkoebsservice/2026-05-19-aarhus-kommune-reducerer-microsoft-afhaengighed-med-25-proce

- Aarhus: whole-stack review, not just office software
- Aarhus: 25% open source office software target by 2030
- Ishøj: only ∼50% of staff now hold a full M365 license

# Denmark: Sovereignty Reaches the Government Platform



## Klumme: Digital suverænitet begynder i udbudsfasen

Det er ikke nok med gode intentioner og hensigtserklæringer. Digital suverænitet kræver effektive tiltag i stor skala. Vi kan med fordel lade os inspirere af EU.

"Danmark skal ud af sin stavnsbinding til big tech og have større digital selvbestemmelse"
– Morten Kjærsgaard, Magenta, ITWatch

Source: https://itwatch.dk/Klummer/article19459678.ece

- New minister: Christina Egelund (M)
- "Digital sovereignty" mentioned six times in the government platform
- Political recognition is ahead of concrete follow-through

# Digital Sovereignty – Digital suverænitet



Digital suverænitet (også kaldet it-suverænitet, cybersuverænitet) betyder, at aktøren har eksklusiv selvbestemmelse (kontrol) over sine egne data og it-infrastruktur (incl. it-tjenester, datanettet). Med eksklusiv selvbestemmelse menes, at aktøren har de evner og færdigheder, der er nødvendige for at opfylde sine roller autonomt, uafhængigt og sikkert.[1][2]

- Åbne og frie dataformater og digitale grænseflader (incl. APIere).
- Dataforvaltningsplan - forudsætter skabelse af metadata, der kan sikre at data kan genfindes - og fx anvendes maskinelt.
- Free/Libre and Open Source Software, FLOSS, FOSS - kan oversættes til "fri og åben-kildekode software". FLOSS's kildekode er let tilgængeligt ("åbent"), og at det kan bruges, ændres og deles frit ("gratis") grundet de anvendte licenser. Dette muliggør aktørens autonomi og uafhængighed.
- Private cloud - ejes og styres alene af aktøren selv - og private cloud er i samme land som aktøren.
- Open-source hardware (en) - dvs designet er let tilgængeligt ("åbent"), og at det kan bruges, ændres og deles frit ("gratis").

Source: https://da.wikipedia.org/wiki/Digital_suver%C3%A6nitet - On purpose taking the danish wording

# My Intentions Today

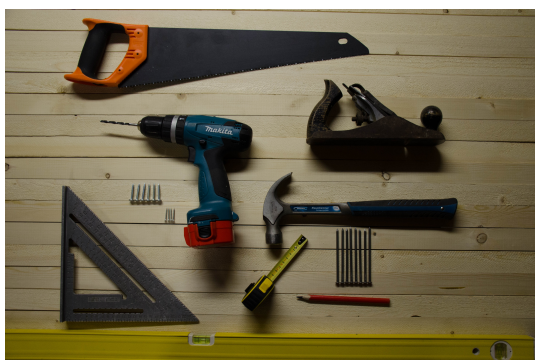


> After the course, you'll be able to get started hosting your own web services and gain hands-on experience managing the infrastructure behind them. You'll get a realistic overview of what it takes – technically, practically, and from a security standpoint – to take control of your own data.

I want to inspire you to change your minds and get started

- What is it?! You are unsure of how to do it
- My intention is to be your sparring partner today, list the parts I think must be considered.
- a practical introduction to self-hosting and digital sovereignty: manage your infrastructure, how to run your own services, and protect your data.
- The slideshow can hopefully serve as an inspiration, but maybe also a Warning! – sometimes let others do the technical stuff ☺

# Why are we here today then?!



- Identify what we *need* for Self-hosting and Digital Sovereignty
- Consider more parts of the picture, a strategy for moving into this
- Show solutions that have worked for me, with some example technologies

Keywords: hosting, self-hosting, digital sovereignty, infrastructure, web solutions, containers, security

# Where Does Your Service Run?



From fully managed cloud to bare metal in your own rack – choosing a hosting model means trading off control, cost, complexity, and trust.

- **SaaS** – Software as a Service: vendor runs everything, just use it (Gmail, Office 365, Salesforce) – no ops, but no control
- **PaaS** – Platform as a Service: deploy your code, vendor manages the runtime, scaling, and OS (Heroku, Render, fly.io)
- **IaaS** – Infrastructure as a Service: you get virtual machines, you manage everything above the hypervisor (AWS EC2, Hetzner Cloud, DigitalOcean)
- **VPS** – a cheap, simple IaaS instance; the typical entry point for self-hosters
- **On-premises / self-hosted** – your own hardware, your own network, full control and full responsibility
- **Hybrid** – mix of the above; e.g. data on-prem, compute in the cloud
- Key questions: Who owns the data? Who is on call at 3am? What is the ongoing cost, price increase next-year, and exit cost?

# Self-hosting

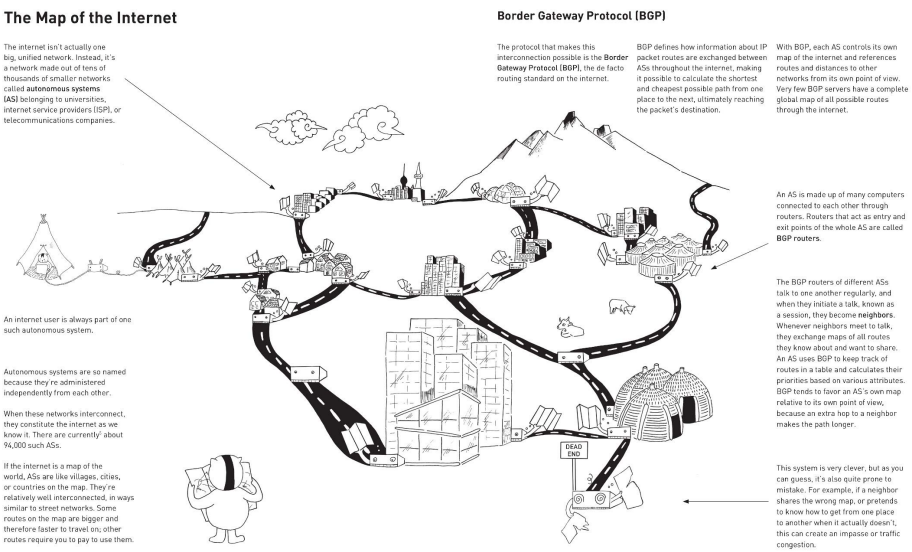


> Self-hosting is the practice of running and maintaining a website or service, as well as own servers for e-mail, IM, NTP and so on, using a private server, instead of using a service outside of the administrator's own control. Self-hosting allows users to have more control over their data, privacy, and computing infrastructure, as well as potentially saving costs and improving skills.[1][2]

Source: https://en.wikipedia.org/wiki/Self-hosting_(network)

- Benefits and Downsides! Which is exactly what we are talking about today
- Take small steps is my recommendation!
- What is most important the data, data format, or where it is hosted?

# What the internet is actually made up of




Source: *How the Internet Really Works An Illustrated Guide to protocols, privacy, censorship, and governance* by catnip and article 19 https://catnip.article19.org/ ❤ fun and educational book

# What is the Internet



RFC-1958:

> A good analogy for the development of the Internet is that of constantly renewing the individual streets and buildings of a city, rather than razing the city and rebuilding it. The architectural principles therefore aim to provide a framework for creating cooperation and standards, as a small "spanning set"of rules that generates a large, varied and evolving space of technology.

Communication between humans - currently!

Based on TCP/IP

- best effort
- packet switching (IPv6 calls it packets, not datagram)
- *connection-oriented* TCP
- *connection-less* UDP

## Internet is Open Standards!



We reject kings, presidents, and voting.
We believe in rough consensus and running code.
– The IETF credo Dave Clark, 1992.

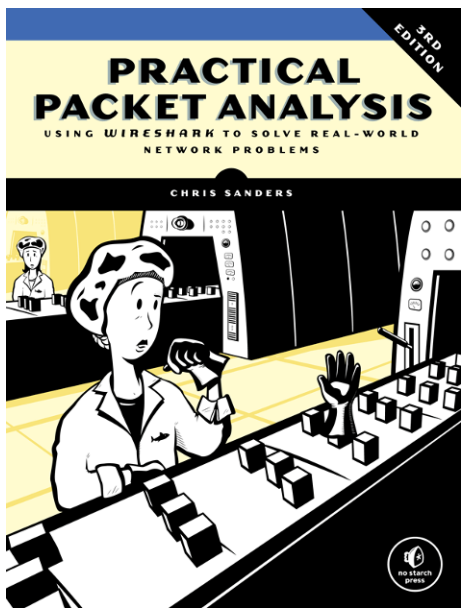
Request for comments (RFC) – a series of documents spanning decades

RFC, BCP, FYI, informational – first ones from 1969!

Are not updated but status is changed to Obsoleted when new versions are published

Standards track:
Proposed Standard $\rightarrow$ Draft Standard $\rightarrow$ Standard

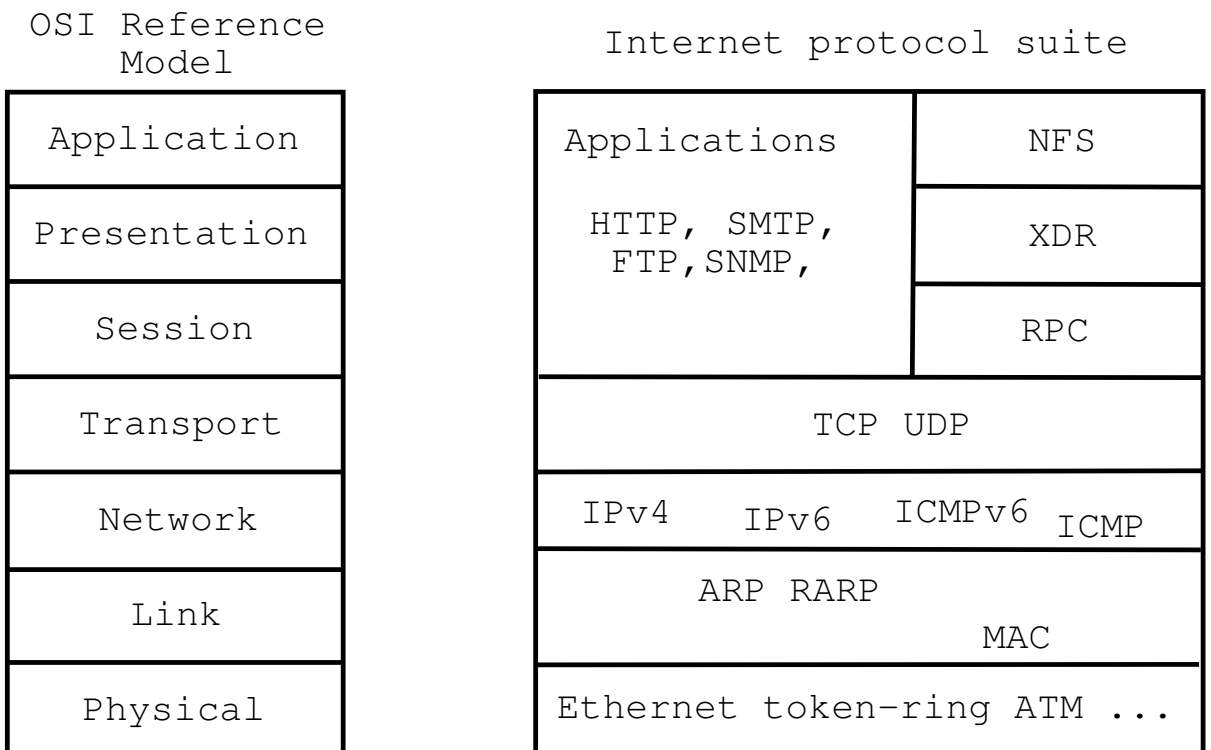
# Practical Packet Analysis




- *Practical Packet Analysis - Using Wireshark to Solve Real-World Network Problems*, 3rd edition 2017, 368 pp Chris Sanders ISBN: 9781593278021

# OSI Model and Internet Protocols





# Recommended network technologies to learn



Small set of recommended networking technologies to learn

Networking: Basic Protocols from the Internet Protocols suite IP/TCP, or TCP/IP

- Network Layer: Ethernet, Address Resolution Protocol (ARP), IPv4 and ICMP
  Later add Wi-Fi and IPv6
- Transport Layer: Transmission Control Protocol (TCP) and User Datagram Protocol (UDP)
- Common upper layer: Dynamic Host Configuration Protocol (DHCP), Domain Name System (DNS), Hypertext Transfer Protocol (HTTP)
  Later add the encrypted/secure versions like Hypertext Transfer Protocol Secure (HTTPS) which uses Transport Layer Security (TLS)

Enough to be able to go to other resources, and a connected path from ARP to TLS

Great chapter on DNS!

# Unbound and NSD



Unbound is a validating, recursive, caching DNS resolver. It is designed to be fast and lean and incorporates modern features based on open standards.

To help increase online privacy, Unbound supports DNS-over-TLS which allows clients to encrypt their communication. In addition, it supports various modern standards that limit the amount of data exchanged with authoritative servers.

https://www.nlnetlabs.nl/projects/unbound/about/

My preferred local DNS server, and NSD is my preferred authoritative DNS server

# Initial advice for network security – especially when hosting



> Quad9 is subject to Swiss privacy law, and the Swiss government extends that protection of the law to Quad9's users throughout the world, regardless of citizenship or country of residence.

- Use a malware filtering DNS service, like Quad9 https://quad9.net/
  point your internal DNS servers to it, and add filtering and logging
- Portscanning - start using portscans in your networks, verify how far malware and hackers can travel, and identify soft systems needing updates or isolation
- Have separation – anywhere, starting with organisation units, management networks, server networks, customers, guests, LAN, WAN, Mail, web, ...
- Use Web proxies - do not allow HTTP directly except for a short allow list,
  do not allow traffic to and from any new TLD
- Only allow SMTP via your own mail servers, create a simple forwarder if you must
- In general *harden your systems* – devices with secure configuration are harder to hack

# Routing Basics: Prefixes, AS and BGP



- **Prefix** – a block of addresses, like 192.0.2.0/24 or 2001:db8:1234::/48
- **Autonomous System (AS)** – a network under one routing policy, identified by an AS number (ASN) registered at an RIR such as RIPE NCC
- **BGP** – Border Gateway Protocol, how autonomous systems tell each other which prefixes they can reach
- **RPKI and ROA** – signed statements of which AS may originate a prefix, prefixes without a valid ROA are filtered by many networks
- **Upstream / transit** – the provider that announces your prefixes to the rest of the internet
- **Smallest routable prefix** – normally /24 for IPv4 and /48 for IPv6
- Using your own PI addresses requires an ASN, a BGP session and an upstream willing to announce them – usually a business connection.
- Exception: Netnørden offers BGP on private connections https://netnoerden.dk/faq

# Public IP address: Provider Aggregatable (PA) Addresses



> PA addresses are assigned from an LIR's allocation and are registered in the RIPE Database by the LIR. The advantage of PA addresses is that the routing information for many customers can be aggregated once it leaves the provider's routing domain.

Source: RIPE Network Coordination Centre (RIPE NCC) – FAQ: ISPs
https://www.ripe.net/membership/member-support/faqs/isp-related-questions/pa-pi/

- Assigned from a Local Internet Registry's (LIR) own allocation
- Registered in the RIPE Database by the LIR itself
- Routing information for many customers can be **aggregated** into a single announcement, improving global routing efficiency
- If an end user changes their upstream provider, the PA addresses **must be returned** – they cannot be taken along
- **Usually one IPv4 for your router, or shared NAT (CGNAT) – you are here**

# Public IP address: Provider Independent (PI) Addresses



> PI address space is assigned separately and not from an LIR's PA allocation. All PI assignments are registered in the RIPE Database by the RIPE NCC at the time they are assigned. PI assignments are usually small; they cannot be aggregated into larger blocks. The disadvantage of this is that network operators throughout the Internet may choose not to route them.

Source: RIPE Network Coordination Centre (RIPE NCC) – FAQ: ISPs
https://www.ripe.net/membership/member-support/faqs/isp-related-questions/pa-pi/

- Assigned **directly by the RIPE NCC** (a Regional Internet Registry), not from an LIR's allocation
- Registered in the RIPE Database by the RIPE NCC at assignment time
- Usually small blocks that **cannot be aggregated** into larger ones
- The end user **retains** the address block even when changing ISPs
- Network operators across the Internet may choose **not to route** PI assignments due to lack of aggregation

# Current Status IP addresses IPv4 vs IPv6

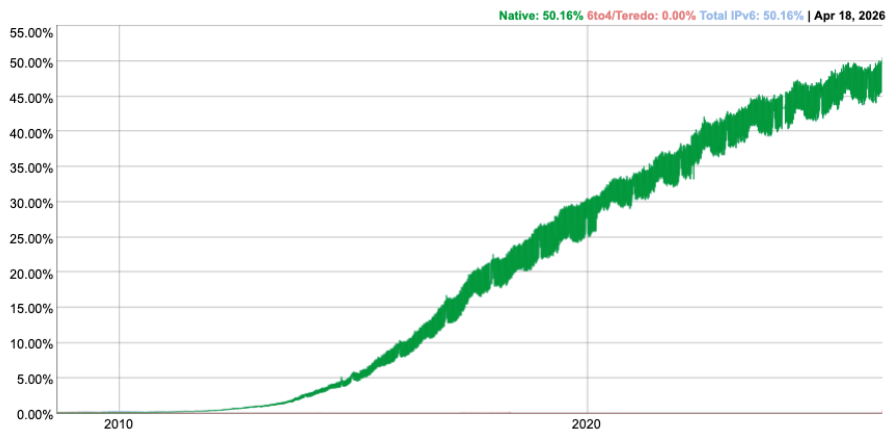


As the RIPE NCC has run out of IPv4 addresses, no new IPv4 PI address space can be assigned. IPv6 PI address space is still available.

- **IPv4 PI:** No longer available – RIPE NCC has exhausted its IPv4 pool, can be bought around 20-40USD/IP in chunks of at least 256
- **IPv4 PA:** Limited availability – RIPE NCC has exhausted its IPv4 pool, providers are charging per IP
  Usually in Denmark you would get a single public IP for your home connection and enterprise customers can pay for more
- **IPv6 PI: Still available for assignment through a sponsoring LIR**
- **IPv6 PA:** Available through LIR – but cannot be migrated to new provider!

So yes, I consider IPv6 a pre-requisite for digital sovereignty!

# IPv6 in Denmark and the EU




"Now, 18 years since Google started recording this data, access via native IPv6 has for the first time exceeded 50% (50.10% on 28 March 2026, to be precise)."

Source: https://pulse.internetsociety.org/en/blog/2026/04/18-years-later-ipv6-reaches-majority/

# The cost of public IP addresses



RIPE NCC members pay an annual contribution (service fee) per Local Internet Registry (LIR). For 2026, this fee is €1,800.
This is charged on a pro rata basis for new members. New members and existing members registering additional LIR accounts also pay a sign-up fee, which is currently €1,000.
Members can have more than one LIR account, but they must pay the membership and sign-up fee for each LIR account. Members also pay €75 per independent assignment or legacy resource and €50 per ASN assignment.

Source: https://www.ripe.net/membership/payment/

- Your own PA allocation requires becoming a RIPE NCC member (LIR) – expensive
- IPv6 PI is cheap, even a private person can request a /48 through a sponsoring LIR for about EUR 100 per year
- Using PI still requires an ASN and an upstream that runs BGP with you

Full-disclosure: I have a company that does these requests and charge a yearly fee of €100 – where €75 is paid to RIPE

# What is a Home Lab



> "A homelab is a self-contained IT environment that you design and control. It usually includes basic infrastructure like compute, storage, and networking, along with a hypervisor or virtualization layer. Some setups are modest and run on a single machine, while others resemble mini datacenters with multiple nodes and shared storage such as SAN or NAS."

Source: StorMagic, "What Is a Homelab? Why IT Pros Are Building Their Own"(stormagic.com, 2025)

- it's your own private sandbox – no stakes
- no risk to production systems
- where you can freely experiment with servers, networking, virtualization, security tools,
- and anything else you're curious about
- I think that doing any kind of experiment, then you *have a lab*

Note: yes, you can run your services at home, but I definitely prefer hosting things outside of my home!

# Hosting at Home: Pros



- Full control and privacy – data stays on hardware you own, with no third party scanning or selling it
- Cost savings over time – after upfront hardware costs, no ongoing expenses or monthly SaaS fees
- Deep customization – configure every service exactly as you want, with no vendor-imposed limitations
- Upgrade to new versions when you want to, or not if you like the long term stable version
- No vendor lock-in – immune to price hikes, shutdowns, or feature removals by external providers
- Learning technologies – hands-on experience with networking, Linux, and server security

# Hosting at Home: Cons



- If using IPv4 – you only have one IP, IPv6 may or may not be available
- Reliability risk – home power lack data-centre-grade battery backup (UPS)
- Security burden – misconfigured or unpatched services exposed to the internet can put your whole network at risk
- Email delivery is hard – residential IPs are often blocklisted; SPF/DKIM/DMARC setup is non-trivial
- ISP restrictions – port 25 blocking and dynamic IPs complicate running mail or web servers
- You have a crap router on home connections, even on fiber connections with 1Gbps!
- No built-in redundancy – hardware failure, fire, or flood can mean permanent data loss without disciplined offsite backups

# Discussion: Where Should Your Services Run?



- Name one service you use today – would you move it home, to a Danish/EU provider, or leave it where it is?
- Does it need to be reachable from the internet, or is VPN access enough?
- How long can you live without it if it breaks – an hour, a day, a week?
- Who fixes it when it breaks while you are on holiday?
- Your VPS is in Frankfurt, but the company is American – whose law applies to your data?
- Where is the offsite copy of your backup, and when did you last restore from it?

# Break




Photo by Alexas Fotos on Unsplash

# Session 1: Technologies and securing your environment



# Operating Systems



Computers need software – operating systems, and applications run on an OS:

- OS is the base for your experiments
- Must be managed, installed at least, updated and packages installed on top
- Some applications need a regular OS, others are fine with containers
- Some applications like Python can even use a virtual environment

TL;DR you need even more skills, sorry!

# Virtualization

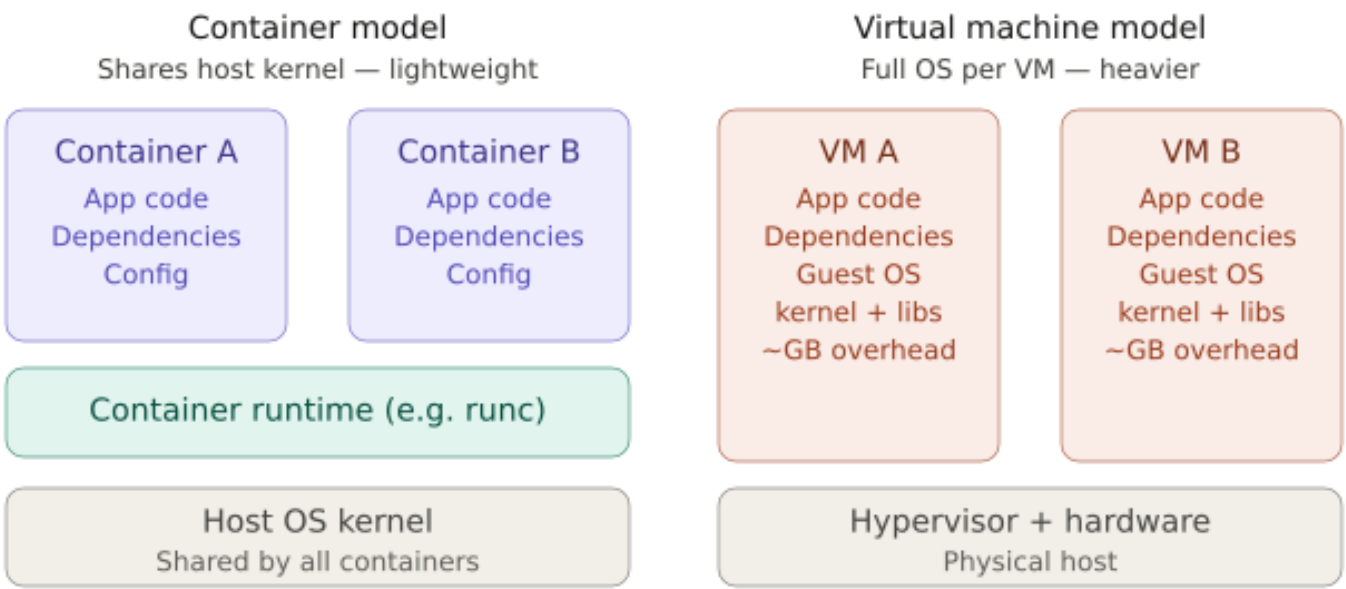


Computers can run multiple operating systems, and OS distributions at the same time:

- Full virtual machine virtualization
- Containers – including containers platforms like Kubernetes
- Use it, and create many virtual machines/containers
- Switch context easily, one project - one (or more) VMs
- Create a new VM in seconds – keep one template VM updated that you can clone
- Install crap – delete a VM, retry
- Performance of VMs and containers on modern CPUs is great! Almost like having two physical computers!

Good news, you can do it all in a single PC! Even a small one!

# Virtualization Overview



# Example software for virtualisation



- **KVM** (Kernel-based Virtual Machine) – Linux kernel hypervisor, often managed via libvirt/virt-manager
- **Proxmox VE** – Open-source platform combining KVM and LXC containers, very popular for home labs
- **Microsoft Hyper-V** – Built into Windows Server and Windows 10/11 Pro, easy entry point for Windows users
- **VirtualBox** – Free, cross-platform Type-2 hypervisor by Oracle, great for desktop-level home labs
- **VMware ESXi** – Enterprise-grade Type-1 hypervisor with a free tier (though licensing has changed under Broadcom)
- **XCP-ng** – Open-source fork of Citrix Hypervisor (XenServer), managed via Xen Orchestra

I use KVM for labs and production, you need to evaluate which one to use

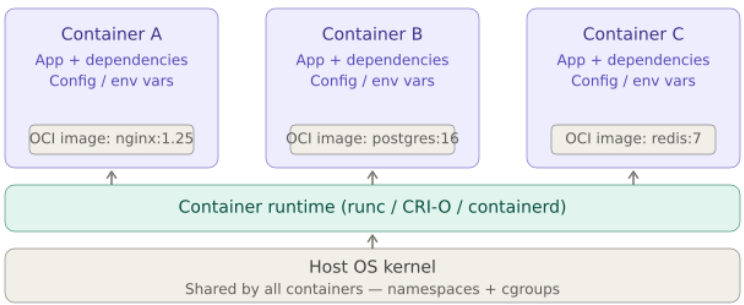
# Virtual Private Servers (VPS)



- A VPS is a virtualised slice of a physical server, giving you dedicated CPU, RAM, and storage
- Runs its own full operating system – you have root access and full control over the environment
- Provided by hosts such as DigitalOcean, Linode, and Hetzner at low monthly cost
- Ideal for hosting websites, mail servers, and small applications with predictable traffic
- Scales vertically by upgrading the plan, but requires manual intervention to resize
- Similar to running a virtual machine on your laptop
- Full system administration, like Debian `apt update; apt upgrade`

Note: the three vendors listed have been used by my customers, and is not a recommendation per se. Evaluate options carefully for yourself

# Containers




- Containers package an application and all its dependencies into a single portable unit
- Share the host OS kernel – lighter and faster to start than full virtual machines
- Docker is the dominant developer-facing tool; images are built from a `Dockerfile` and stored in registries
- Images are not Docker-specific – the industry standard is the **OCI image format**, supported by all modern tools
- **Compose files** allow multi-container applications (e.g. app + database) to be defined and run together
- Pre-packaged systems that require less administration – **only configuration**

# Container Standards and Runtimes



- The **Open Container Initiative (OCI)** defines three open standards: image-spec, runtime-spec, and distribution-spec https://opencontainers.org
- Any OCI-compliant image can be built with one tool (e.g. Buildah, Docker) and run with another – no vendor lock-in
- **runc** is the reference low-level runtime implementing the OCI runtime-spec; most runtimes use it under the hood
- Kubernetes communicates with runtimes via the **Container Runtime Interface (CRI)** – a gRPC-based plugin API
- **containerd** and **CRI-O** are the two main CRI-compliant runtimes; Docker's direct integration was removed in Kubernetes v1.24 https://kubernetes.io/docs/setup/production-environment/container-runtimes/
- CRI-O is purpose-built for Kubernetes – lightweight, OCI-compliant, and the default in Red Hat OpenShift https://cri-o.io

# Kubernetes



## Run K8s Anywhere

Kubernetes is open source giving you the freedom to take advantage of on-premises, hybrid, or public cloud infrastructure, letting you effortlessly move workloads to where it matters to you.

To download Kubernetes, visit the download section.

- Kubernetes (K8s) is an open-source system for automating deployment and management of containers at scale
- Organises containers into *Pods*, grouped by *Deployments*, and exposed via *Services*
- Provides automatic scaling, self-healing, and rolling updates with zero downtime
- Suited to large, complex microservice architectures – significant operational overhead for small projects
- Managed offerings (GKE, EKS, AKS) reduce setup burden but increase cloud dependency

# Summary: Technology Comparison



| Technology | Best For | Complexity | Scalability |
|---|---|---|---|
| VPS | Single apps, full OS control | Low | Vertical (manual) |
| Containers | Portable, consistent deploys | Medium | Horizontal (manual) |
| Kubernetes | Large-scale microservices | High | Horizontal (auto) |
| Virtual Hosts | Multi-site on one server | Low | Limited |

- Above does not consider if you are a private person running systems for yourself, friends, family
- If you are a company with a few services
- If you are a large company that want to switch over to something privately owned
- Above are in my opinion good options for running your complete infrastructure!

# Virtual Hosts in Web Servers



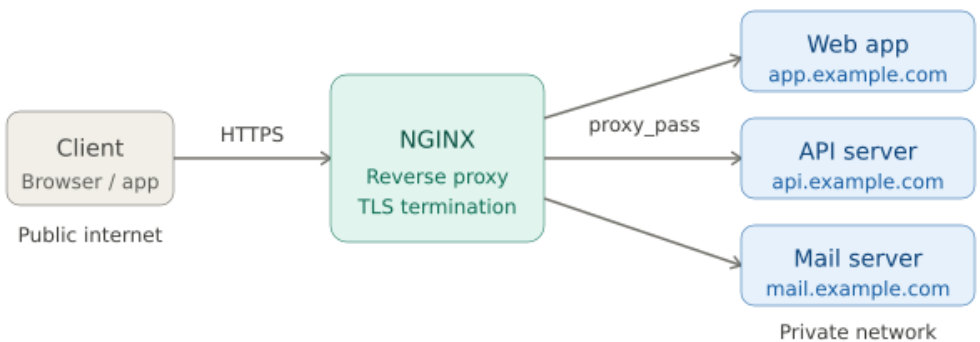
```
server { server_name example.com www.example.com; root /var/www/example.com; }
server { server_name example.org; root /var/www/example.org; }
```

- One server with one IP address can serve many websites
- DNS for both **example.com** and **example.org** points to the same IP
- The browser sends the name it wants in the HTTP **Host** header, the web server picks the matching site
- With HTTPS the name is also sent in TLS as **SNI**, so the server can present the right certificate

Side note: Digital Ocean and NGINX as a reverse proxy

That said DigitalOcean has very good documentation for lots of things! example:
https://www.digitalocean.com/community/tutorials/how-to-configure-nginx-as-a-reverse-proxy-on-ubuntu-22-04

# NGINX as a Reverse Proxy

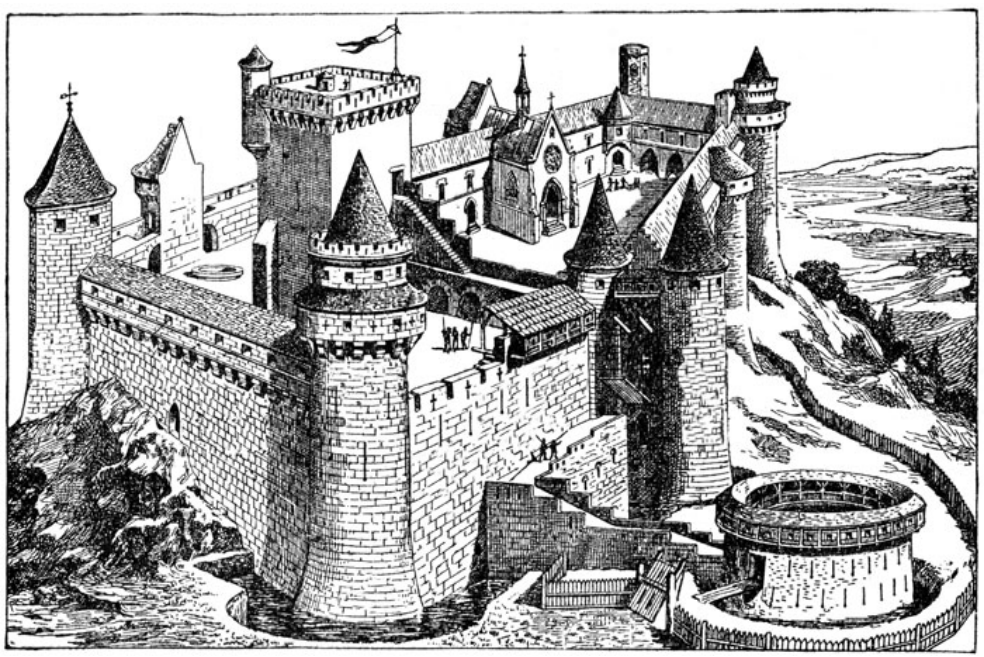



- A reverse proxy sits in front of backend servers, accepting client requests and forwarding them internally
- NGINX uses the `proxy_pass` directive to route traffic to upstream services by hostname or IP
- Can provide TLS termination, load balancing, caching, and request buffering at the edge
  A single system with NGINX get certificates with Let's Encrypt (or alternatives)

# Administration and securing your own environment

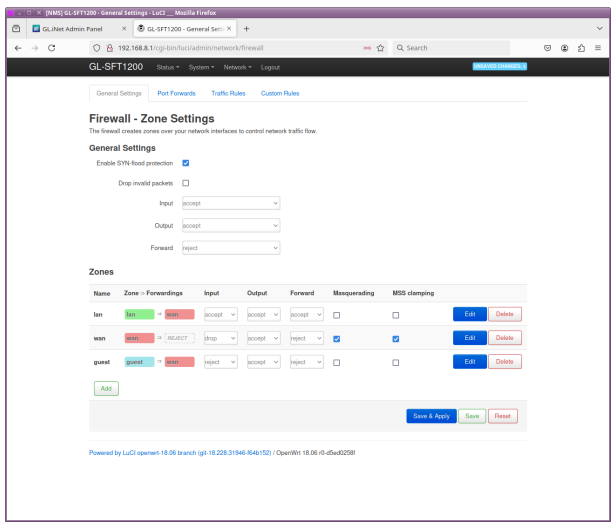


Defense in depth



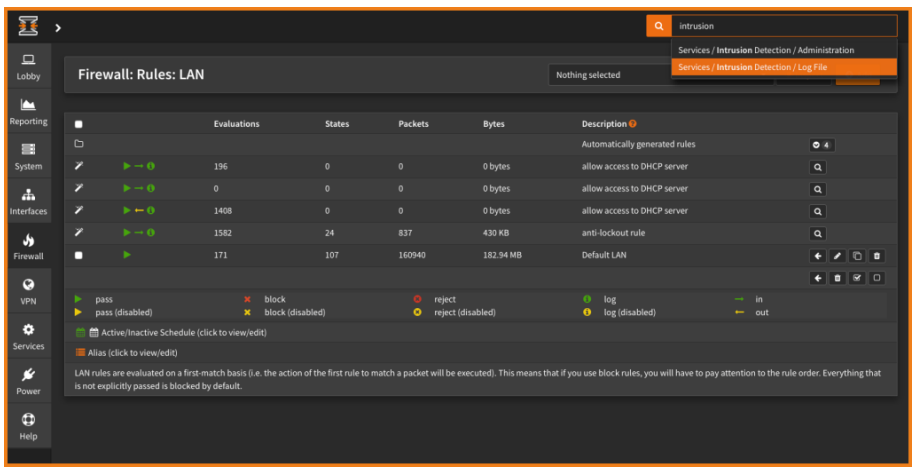
Picture originally from: http://karenswhimsy.com/public-domain-images

# Firewalls




- Firewalls are a great project to look into, there are so many ☺
- I recommend looking into open source firewalls like OPNsense, UFW and OpenBSD and many routers also include firewalls. You can often find images from commercial vendors, that you can try out.
- Picture is from GL-Inet router Opal with OpenWRT/LuCI which is router software for smaller routers

# Firewall OPNsense GUI based and easy to install




- OPNsense https://opnsense.org/
- Firewall built on FreeBSD with web interface
- Danish companies have been using these for many years now

# Linux Uncomplicated Firewall (UFW)



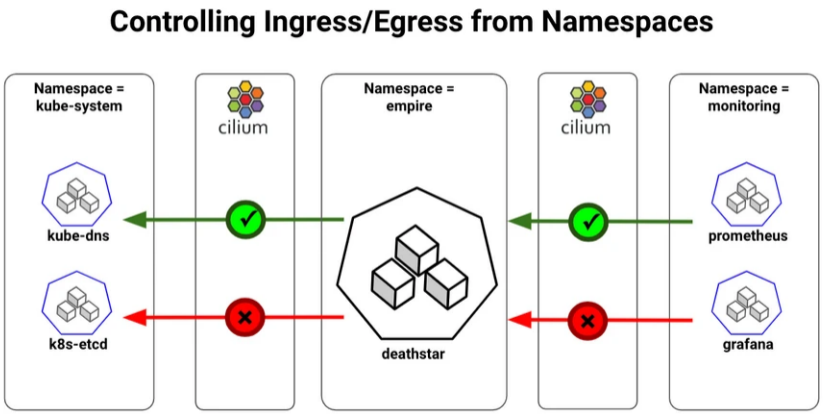
Debian / Ubuntu:

```
root@debian01:~# apt install ufw
...
root@debian01:~# ufw allow 22/tcp
Rules updated
Rules updated (v6)
root@debian01:~# ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
root@debian01:~# ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 22/tcp (v6)                ALLOW IN    Anywhere (v6)
```

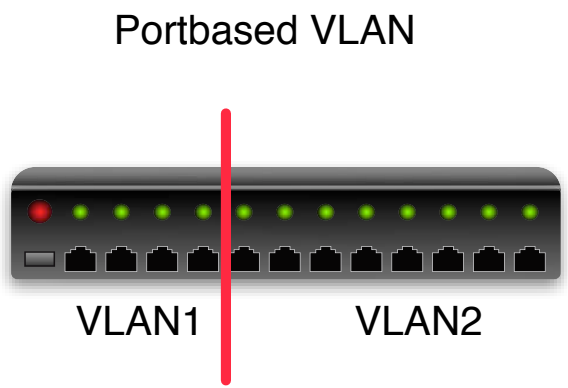
- Extremely easy to use – I recommend and use the (Uncomplicated Firewall) UFW
- Perfect for a VPS, virtual server somewhere

# Kubernetes Network parts – NetworkPolicy




- Container Network Interface (CNI) and Cloud Native Computing Foundation (CNCF) project https://github.com/containernetworking/cni
- Multiple options – popular and common ones Calico, Flannel and Cilium
- Cilium - I have followed Cilium for a while, so I choose that

# Together with Firewalls - Virtual LAN (VLAN)



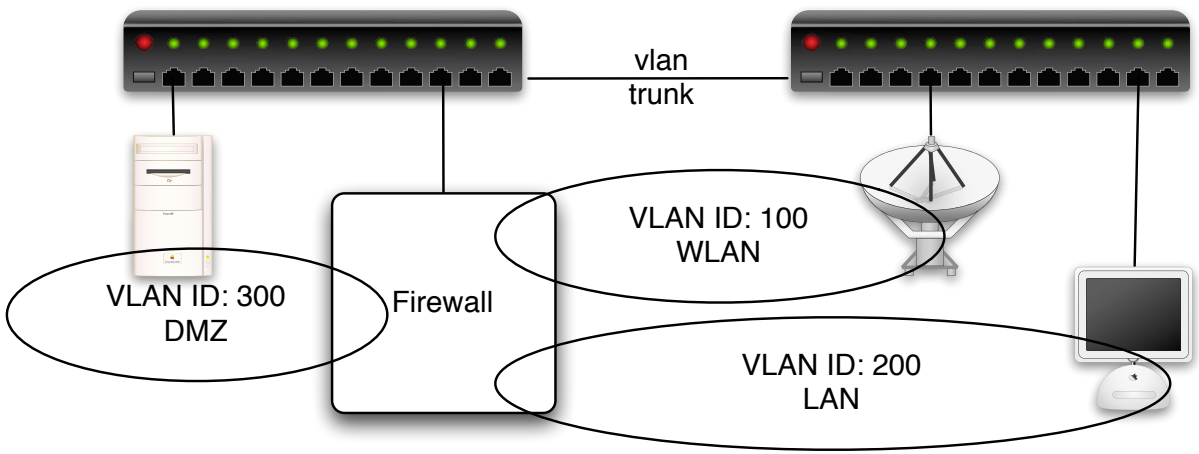

Managed switches often allow splitting into zones called virtual LANs

Most simple version is port based

Like putting ports 1-4 into one LAN and remaining in another LAN

Packets must traverse a router or firewall to cross between VLANs

# Virtual LAN (VLAN) IEEE 802.1q




Using IEEE 802.1q VLAN tagging on Ethernet frames

Virtual LAN, to pass from one to another, must use a router/firewall

Allows separation/segmentation and protects traffic from many security issues

# Network Segmentation: Which VLANs to create



Using VLANs to segment networks we can extend it to the rest of the network, and an example of isolated segments could be:

- Guest network for cabled and Wi-Fi clients
- Client networks can be isolated per department, floor or building
- Server networks for multiple purposes – development, staging, testing, production, manufacturing, ...
- Internet servers in a demilitarized zone (DMZ) which would make it less likely that compromised internet server would affect the rest of the network
- Dedicated printer network
- **Management network for virtualisation, one for network management, one for storage**

# Example, Restricting the Kubernetes API



Watch out when learning, you may expose an *admin interface*

> **By default, the Kubernetes API server listens on port 6443 on the first non-localhost network interface, protected by TLS**. In a typical product ion Kubernetes cluster, the API serves on port 443. The port can be changed with the –secure-port, and the listening IP address with the –bind -address flag.
>
> ...
>
> If your cluster uses a **private certificate authority, you need a copy of that CA certificate configured into your /.kube/config on the cli ent**, so that you can trust the connection and be confident it was not intercepted.

- If an attacker can run Kubectl with credentials – they can do anything
- Defense in Depth is suggested – so restrict at multiple levels
- Only allow connections from specific network – firewall filtering
- Use TLS and verify/copy certificates

# Proxy servers and Web Application Firewalls (WAF)



Filtering at higher layers is also possible

- Web proxies for clients can help security a lot – a centralized filter for everyone
- Reverse proxies for web applications are called Web Application Firewalls (WAF) – and filter incoming web requests, and outgoing answers. Can help with attacks like SQL injection and exfiltration of data
- Depending on your network it can replace or be combined with filtering on DNS servers, and I would prefer to filter domains with DNS
- I would also prefer blocking large prefixes of IP destinations using routers/stateless packet filters – maybe use BGP for distributing *lists* or automation for downloading block lists

Also great home lab projects!

# CrowdSec – Crowd sourced security information



**Detect and block Log4j exploitation attempts with CrowdSec**

If you work in Infosec, you had a very lousy weekend. And that's because of the Log4j zero-day vulnerability (CVE-2021-44228) that was discovered. We had no cho ice but to roll up our sleeves to help our community before things got messier than they already were.

As a result, we have released a scenario that will help you detect and block exploitation attempts of the vulnerability. This new scenario can be directly downl oaded from our Hub and installed in a blink of an eye. Check this quick video to see the plugin in action:

https://www.crowdsec.net/blog/detect-block-log4j-exploitation-attempts

- Log4j is a popular software library in the Java world
- CrowdSec quickly provided a list of systems scanning the internet for this vulnerability
  https://gist.github.com/blotus/f87ed46718bfdc634c9081110d243166
- Full disclosure they gave me a hoodie at an event ☺

# CrowdSec with NGINX



- **CrowdSec** is an open-source, community-driven security engine – the llama company https://crowdsec.net
- **NGINX Bouncer** – a Lua plugin enforcing ban decisions in real time via `access_by_lua_block`, blocking malicious IPs before requests reach the backend
- **Security Engine** – reads NGINX logs and detects behavioural patterns: HTTP probing, scanning, brute force, and matches against the community blocklist
- **AppSec/WAF component** – inline virtual patching, SQLi, XSS, and known CVE protection; install with: `cscli collections install crowdsecurity/appsec-virtual-patching`
- **Community Blocklist** – crowd-sourced IP reputation feed, continuously updated from millions of sensors worldwide; free tier available
- Two complementary modes: reactive (log-based, bans offenders) and inline (WAF, blocks the request before it is processed)

Source: Docs: https://docs.crowdsec.net/u/bouncers/nginx/

# The OWASP Core Rule Set for WAF



- **OWASP Core Rule Set (CRS)** – the standard open source rules, protects against SQL injection, XSS, remote code execution and other common attacks https://coreruleset.org
- CRS uses **anomaly scoring** – each matching rule adds points, the request is blocked when the total passes a threshold. **Paranoia levels** 1–4 – higher levels catch more attacks, but also block more legitimate traffic, start at level 1
- Expect tuning: false positives are normal, run in detection-only mode first and read the logs
- **ModSecurity** – open source Web Application Firewall engine, inspects each HTTP request and response against rules. Works with Apache, NGINX and IIS, version 3 (libmodsecurity) uses a connector module for NGINX
- Trustwave ended support in July 2024, ModSecurity is now an OWASP project https://owasp.org/www-project-modsecurity/
- **Coraza** – newer OWASP WAF engine written in Go, first stable release 2021, fully compatible with CRS v4 https://coraza.io

Note: I am NOT currently running any of these, considering still

# Break




Photo by Alexas Fotos on Unsplash

# Session 2: Operations and running services in practice





# Please do backups, and restore testing!

- **3-2-1 rule:** 3 copies of your data, on 2 different types of storage, 1 copy offsite
  https://en.wikipedia.org/wiki/Glossary_of_backup_terms
- Keep at least one copy offline – ransomware encrypts or deletes the backups it can reach
- RAID, mirrored disks and Git are not backup – a deleted file is deleted on both disks, and a Git repository can be lost or overwritten too
- Encrypt backups that leave your own systems
- A backup you have never restored is not a backup

# Backup tools



A few sample Backup Tools

- **Proxmox Backup Server (PBS)** – Dedicated backup server for Proxmox/KVM environments. Supports deduplication, compression, and incremental backups.
- **Virt-backup** – Lightweight Python tool for backing up libvirt/KVM virtual machines. Simple and scriptable.
- **Restic** – Fast, secure, and deduplicated backups. Supports local, SFTP, S3, and many other backends.
- **Borg Backup** – Deduplicating archiver with compression and encryption. Often paired with Borgmatic for automation.
- **Duplicati** – Web UI-based backup tool with support for cloud storage backends. Good for beginners.
- **Bacula** – Enterprise-grade, client-server backup system. Very flexible but complex to configure.
- **Rsync** – The classic file sync and backup tool. Often used in scripts for simple home lab backups.

# Infrastructure as code



> **Infrastructure as code (IaC)** is the process of managing and provisioning computer data centers through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools.[1] The IT infrastructure managed by this comprises both physical equipment such as bare-metal servers as well as virtual machines and associated configuration resources. The definitions may be in a version control system. It can use either scripts or declarative definitions, rather than manual processes, but the term is more often used to promote declarative approaches.

Source: https://en.wikipedia.org/wiki/Infrastructure_as_code

- Has become the norm in many places
- I use mostly **Ansible** to automate *system administration*
- I highly recommend using version control from the beginning! Use Git to store playbooks

# Docker Compose as Infrastructure as Code



**Docker Compose** is a tool for defining and running multi-container applications using a declarative YAML file – `docker-compose.yml`. The entire application stack, including services, networks, volumes, and environment configuration, is described in a single file that can be version-controlled, reviewed, and reproduced identically on any machine with Docker installed.

Source: https://docs.docker.com/compose/

- A Compose file defines **services** (containers), **networks**, and **volumes** declaratively
- Running `docker compose up` brings up the entire stack – no manual steps
- Files live in version control alongside application code – changes are auditable
- Ideal for development environments, CI pipelines, and simple self-hosted deployments
- Compose files are valid IaC: they are machine-readable, repeatable, and diff-able

# Kubernetes as Infrastructure as Code



> **Kubernetes** manages containerised workloads through declarative YAML manifests. Rather than issuing imperative commands, operators describe the **desired state** of the cluster – deployments, services, ingress rules, storage claims – and Kubernetes continuously reconciles the actual state to match. These manifests are plain text files that belong in version control, making the entire infrastructure reviewable, auditable, and reproducible.

Source: https://kubernetes.io/docs/concepts/overview/working-with-objects/

- Core objects: `Deployment`, `Service`, `ConfigMap`, `Secret`, `PersistentVolumeClaim`
- `kubectl apply -f manifest.yaml` converges the cluster toward the declared state
- **Helm** charts package and template manifests for reusable, parameterised deployments https://helm.sh
- **GitOps** tools such as Flux and ArgoCD apply manifests automatically from a Git repository, making Git the single source of truth for cluster state

# Getting started with Kubernetes: Rancher Desktop



- Open source desktop application from SUSE for containers and Kubernetes on your own computer https://rancherdesktop.io
- Runs on Linux, macOS and Windows – uses a virtual machine on Linux and macOS, WSL2 on Windows
- Choose the Kubernetes version and container engine, containerd or Docker, in a graphical interface
- Includes the usual tools: `kubectl`, `helm`, `docker` and `nerdctl`
- Reset Kubernetes to default with a single click – break things and start over

After installing, check that the local cluster is running:

```
$ kubectl get nodes
$ kubectl get pods -A
```

# Kubernetes with a few containers running



```
hlk@alice:~$ kubectl get node
NAME                   STATUS   ROLES           AGE   VERSION
lima-rancher-desktop   Ready    control-plane   74d   v1.36.3+k3s1
hlk@alice:~$ kubectl get pod -A
NAMESPACE     NAME                                      READY   STATUS      RESTARTS       AGE
default       juice-shop-86c67bcdf4-9mgsv               1/1     Running     4 (49s ago)    39d
juice-shop    juice-shop-86c67bcdf4-8gfsl               1/1     Running     2 (49s ago)    28d
juice         juice-shop-86c67bcdf4-tfbrm               1/1     Running     4 (49s ago)    38d
kube-system   coredns-54996dc9b4-l9bdr                  1/1     Running     8 (49s ago)    74d
kube-system   helm-install-traefik-7f9st                0/1     Completed   2 (74d ago)    74d
kube-system   helm-install-traefik-crd-6nrl6            0/1     Completed   0              74d
kube-system   local-path-provisioner-58d557dc48-w4lwz   1/1     Running     8 (49s ago)    74d
kube-system   metrics-server-6dc596dfb8-j2fx8           1/1     Running     8 (49s ago)    74d
kube-system   svclb-traefik-3ac04683-6r98x              2/2     Running     16 (49s ago)   74d
kube-system   traefik-59b7647586-p9cqv                  1/1     Running     8 (49s ago)    74d
hlk@alice:~$
```

Example show OWASP JuiceShop running as a container

# Get ready, Up and running with Ansible



Prequisites for Ansible - you need a Linux machine:

- python language - Ansible uses this
- ssh keys - remote login without passwords
- Become/Sudo/Doas - allow regular users to do superuser tasks
- Recommended tool: `ssh-copy-id` for getting your key on new server
- Recommended Change: `sshd_config` - no passwords allowed, no bruteforce
- Recommended to use: jump hosts/ProxyJump in `ssh_config`
- Highly recommended: Git and/or github for version control

Official docs:
https://docs.ansible.com/projects/ansible/latest/getting_started/index.html

# Important notes about tasks



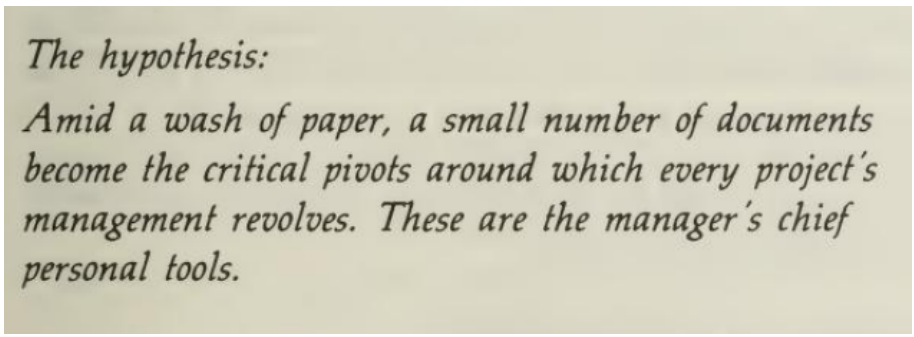
Ansible Tasks are usually **idempotent**. Without a lot of extra coding, bash scripts are usually not safety run again and again. Ansible uses "Facts", which is system and environment information it gathers ("context") before running Tasks.

Ansible uses these facts to check state and see if it needs to change anything in order to get **the desired outcome**. This makes it safe to run Ansible Tasks against a server over and over again.

Also not describing what to do, but what you want the result to be!

Quote from:
https://serversforhackers.com/an-ansible-tutorial but added "usually"since it is easy to mess up with lineinfile tasks, updated link https://serversforhackers.com/c/an-ansible2-tutorial

# Document or it didn't happen

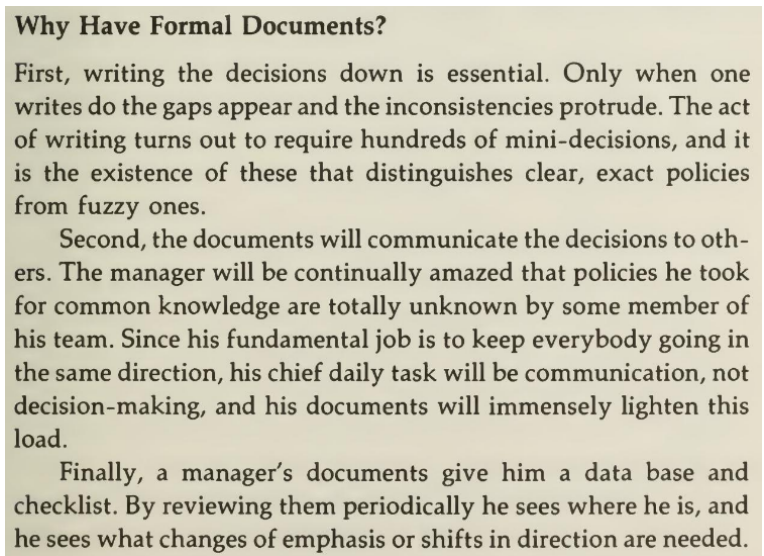


*The hypothesis:*

*Amid a wash of paper, a small number of documents become the critical pivots around which every project's management revolves. These are the manager's chief personal tools.*

- Make sure to document!
- There are so many parts, so many decisions, soo many places thing can mess up
- Have a minimum of documentation – and the YAML files are not enough
- Hint: you can read the Mythical Man Month at https://archive.org/details/MythicalManMonth

# Why Have Formal Documents



**Why Have Formal Documents?**

First, writing the decisions down is essential. Only when one writes do the gaps appear and the inconsistencies protrude. The act of writing turns out to require hundreds of mini-decisions, and it is the existence of these that distinguishes clear, exact policies from fuzzy ones.

Second, the documents will communicate the decisions to others. The manager will be continually amazed that policies he took for common knowledge are totally unknown by some member of his team. Since his fundamental job is to keep everybody going in the same direction, his chief daily task will be communication, not decision-making, and his documents will immensely lighten this load.

Finally, a manager's documents give him a data base and checklist. By reviewing them periodically he sees where he is, and he sees what changes of emphasis or shifts in direction are needed.

Source: *The Mythical Man-Month (Anniversary Edition)* by Frederick P. Brooks Jr.

- I use Git version control for my documentation purposes, some is in Zim Personal Wiki https://zim-wiki.org/
- I also use an IPAM https://spritelink.github.io/NIPAP/

# Document what you did, what it did



```
cilium install --version=1.13.0-rc4 \
               --helm-set ipam.mode=kubernetes --helm-set tunnel=disabled \
               --helm-set ipv4NativeRoutingCIDR="10.0.0.0/8" --helm-set bgpControlPlane.enabled=true \
               --helm-set k8s.requireIPv4PodCIDR=true --helm-set kube-proxy-replacement=strict
  Using Cilium version 1.13.0-rc4
 Auto-detected cluster name: kubernetes
 Auto-detected datapath mode: tunnel
 Auto-detected kube-proxy has not been installed
  Cilium will fully replace all functionalities of kube-proxy
  helm template --namespace kube-system cilium cilium/cilium --version 1.13.0-rc4 --set bgpControlPlane.enabled=true,
cluster.id=0,cluster.name=kubernetes,encryption.nodeEncryption=false,ipam.mode=kubernetes,
ipv4NativeRoutingCIDR=10.0.0.0/8,k8s.requireIPv4PodCIDR=true,k8sServiceHost=10.137.0.26,
k8sServicePort=6443,kube-proxyreplacement=strict,kubeProxyReplacement=strict,operator.replicas=1,
serviceAccounts.cilium.name=cilium,serviceAccounts.operator.name=cilium-operator,tunnel=disabled
...
 Waiting for Cilium to be installed and ready...
 Cilium was successfully installed! Run 'cilium status' to view installation health
```

How I ran the kubeadm for building cluster initially - note the skip:

`sudo kubeadm init --pod-network-cidr=10.50.0.0/16 --skip-phases=addon/kube-proxy`

# Zencurity ApS – Self-Hosted Infrastructure



Datacenter - a small rack setup with:

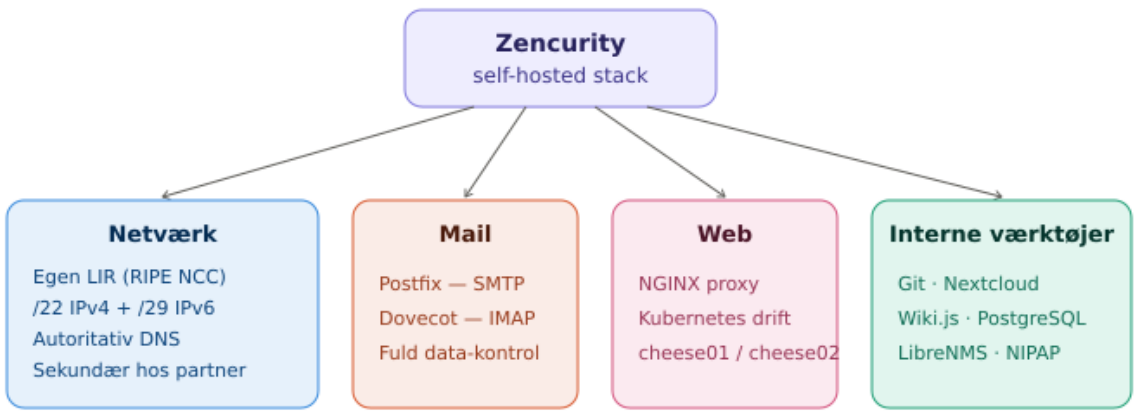
- **Own LIR at RIPE NCC** – assigned a **/22 IPv4** and **/29 IPv6** prefix, fully in control of our address space and routing
- **Authoritative DNS** – self-hosted primary, with a friendly company lending us `ns03` as secondary
- **Email** – **Postfix** for SMTP and **Dovecot** for IMAP, self-hosted with full control over data and delivery
- **Web** – **NGINX** as reverse proxy and web server, with **Kubernetes** for container orchestration of services
- **Virtualisation** – **Debian** with **KVM** as the hypervisor, running virtual machines on our own hardware
- **Guest operating systems** – **Debian** for Linux workloads, **OpenBSD** for security-sensitive and network-facing services – like Wireguard VPN
- Almost everything is version-controlled and automated – infrastructure as code from hypervisor to application layer
- Self-hosting means we own the data, control the stack, and are on call – a deliberate choice
- **Internal services** – self-hosted Git, file sync (Dropbox replacement), PostgreSQL, LibreNMS, NIPAP, Zim-Wiki, and more

# Overview virtualization




Example servers with KVM and Mixed VM and Kubernetes

# Overview services



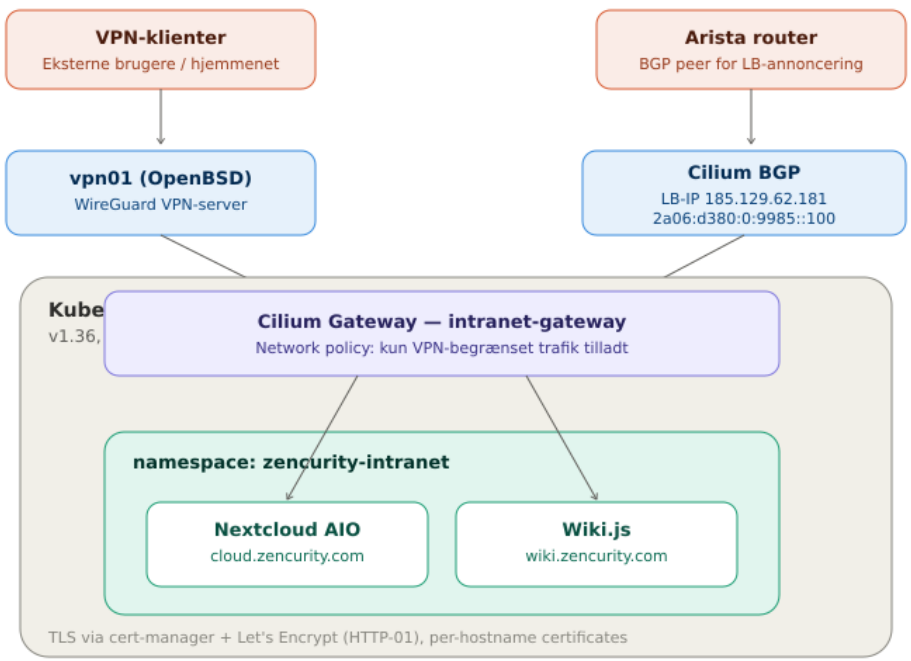

**Fælles for det hele**

Version-kontrolleret og automatiseret — infrastructure as code fra hypervisor til applikationslag

Self-hosting: vi ejer data, styrer stacken, og er selv på vagt — et bevidst valg

Guest-OS: Debian (workloads) og OpenBSD (sikkerheds- og netværksvendte tjenester)

# Kubernetes setup only VPN access to Nextcloud




I do not use all Nextcloud features, so it is only reachable through WireGuard VPN

# Real World: Upgrading Nextcloud on Kubernetes



Critical PHP security advisory – PHP 8.3 to 8.4, Nextcloud 33 to 34, about 30 minutes of work:

- Behind a VPN there is no rush – I could upgrade the next day at a good time, not late in the evening
- Run applications in isolated rooms: own Kubernetes namespace, strict NetworkPolicy with both ingress and egress filters
- Full backup first: a script dumping the 3 databases, and tar of all Nextcloud files
- Check the kernel log with `journalctl -k` for PHP segfaults – nothing found
- Look for new PHP files from the last week – not a guarantee, but cheap to run
- Fetch the latest Helm chart for Nextcloud All-in-One, diff the settings for new required values
- Start the upgrade and wait
- The fulltextsearch app had a missing import, a one-line hotpatch https://github.com/nextcloud/fulltextsearch/issues/1005
- Wait a bit more, done

```
helm upgrade nextcloud-aio nextcloud-aio/nextcloud-aio-helm-chart --version 14.4.1
```

# All Software Has Errors – Know What You Run



- Every package, library and container image will get vulnerabilities – also the ones you forgot you installed
- Scanning tells you what to fix, patching does the fixing – you need both
- Remove what you do not use – fewer packages, fewer findings
- **SBOM** – Software Bill of Materials, a list of everything installed on a system or in an image
- Example tools: **Grype** matches packages against known vulnerabilities, **Syft** builds SBOMs – both open source from Anchore
  https://github.com/anchore/grype
- Debian hosts: the list of installed apt packages goes straight into Grype
- Kubernetes: Syft builds an SBOM for each container image, then Grype scans the SBOM
  The image is only downloaded once per version

# Example: Scanning My Own Infrastructure



- Debian and KVM hosts: a scanner VM collects package lists from 19 hosts over SSH, using a restricted user that can only run a read-only export
- Grype runs nightly, only High/Critical with a fix available
- Push notification with ntfy, only when something new shows up
- First run: 8367 findings – after removing old leftover packages: 827
- 37 Kubernetes images: 1274 findings, nginx:alpine went from 117 to 3 just by pulling the current image
- Floating tags like `latest` bring fixes but change without review – pinned versions are controlled but go stale
- Do not take a new version the minute it is published – most malicious versions are found within days

# Installed Is Not the Same as Running



- The scanner sees what is **installed** – a patched kernel package looks clean, but the old kernel is still **running** until you reboot
- Simple check: compare the running kernel with the newest installed one
- If they differ, the host needs a reboot – the nightly scan sends a notification
- Same problem for libraries: a running service keeps using the old version until it is restarted, `needrestart` on Debian can show which
- Kubernetes nodes: kured reboots nodes one at a time, inside a reboot window

The dpkg log shows the install time, and the last boot shows when the new kernel actually started running:

```
for h in cheese01 cheese02 gouda01; do
  echo "== $h"
  ssh $h 'zgrep -h " install linux-image-6.12.113" /var/log/dpkg.log*; uptime -s'
done
```

# OpenBSD hosts are not scanned



We just update them all with Ansible:

```
for GROUP in openbsd dns
do
  ansible -m syspatch -b --become-method doas $GROUP
  ansible -m shell -a "pkg_add -u"  -b --become-method doas $GROUP
  ansible -m shell -a "fw_update"  -b --become-method doas $GROUP
done
```

These are production systems like mail and name servers!

- You are the system administrator – be prepared to write scripts and automation like this
- Even though some systems automate themselves it is work that **MUST** be done

# Start Here: Own Your Domain and Email



Owning your domain is the single most important first step toward digital sovereignty.

- **Register your own domain** – pick a registrar you trust, pay the yearly fee
- **DNS is the foundation** – controlling your own zone means you control where email, web, and all other services point, and you can move them at any time
- **Use your own domain for email** – `you@yourdomain.com` instead of `you@gmail.com`
- **MX records** point your domain's email to whichever mail server you choose – self-hosted Postfix/Dovecot?
- **SPF, DKIM, and DMARC** are mandatory – without them your outgoing mail lands in spam; DNS records
- **Practical path:** register domain → set up DNS → point MX at a trusted mail host → migrate email → self-host when ready
- To get started perhaps register some new domain to try it out! I use https://Kramse.surf for testing.

# Simple Mail Transfer Protocol (SMTP)



> SMTP is the standard protocol for sending email between servers. Defined in RFC 821 (1982) and updated by RFC 5321 (2008) as Extended SMTP (ESMTP). All mail servers use it – sending, relaying, and receiving between domains.

Source: https://en.wikipedia.org/wiki/Simple_Mail_Transfer_Protocol

- **Port 25** – server-to-server SMTP (often blocked by ISPs on home connections)
- **Port 587** – submission port for mail clients to send via their mail server
- **Port 465** – SMTPS, submission over TLS
- **Postfix** is the most widely deployed open source SMTP server https://www.postfix.org
- **Dovecot** handles IMAP/POP3 – the retrieval side; pairs naturally with Postfix https://www.dovecot.org
- **SPF, DKIM, DMARC** are DNS-based authentication layers built on top of SMTP – without them, your mail will be rejected or marked as spam
- Self-hosting email is rewarding but demanding – start with a VPS with a clean IP reputation

# Internet Message Access Protocol (IMAP)



In computing, the Internet Message Access Protocol (IMAP) is an Internet standard protocol used by email clients to retrieve email messages from a mail server over a TCP/IP connection.[1] IMAP is defined by RFC 9051.

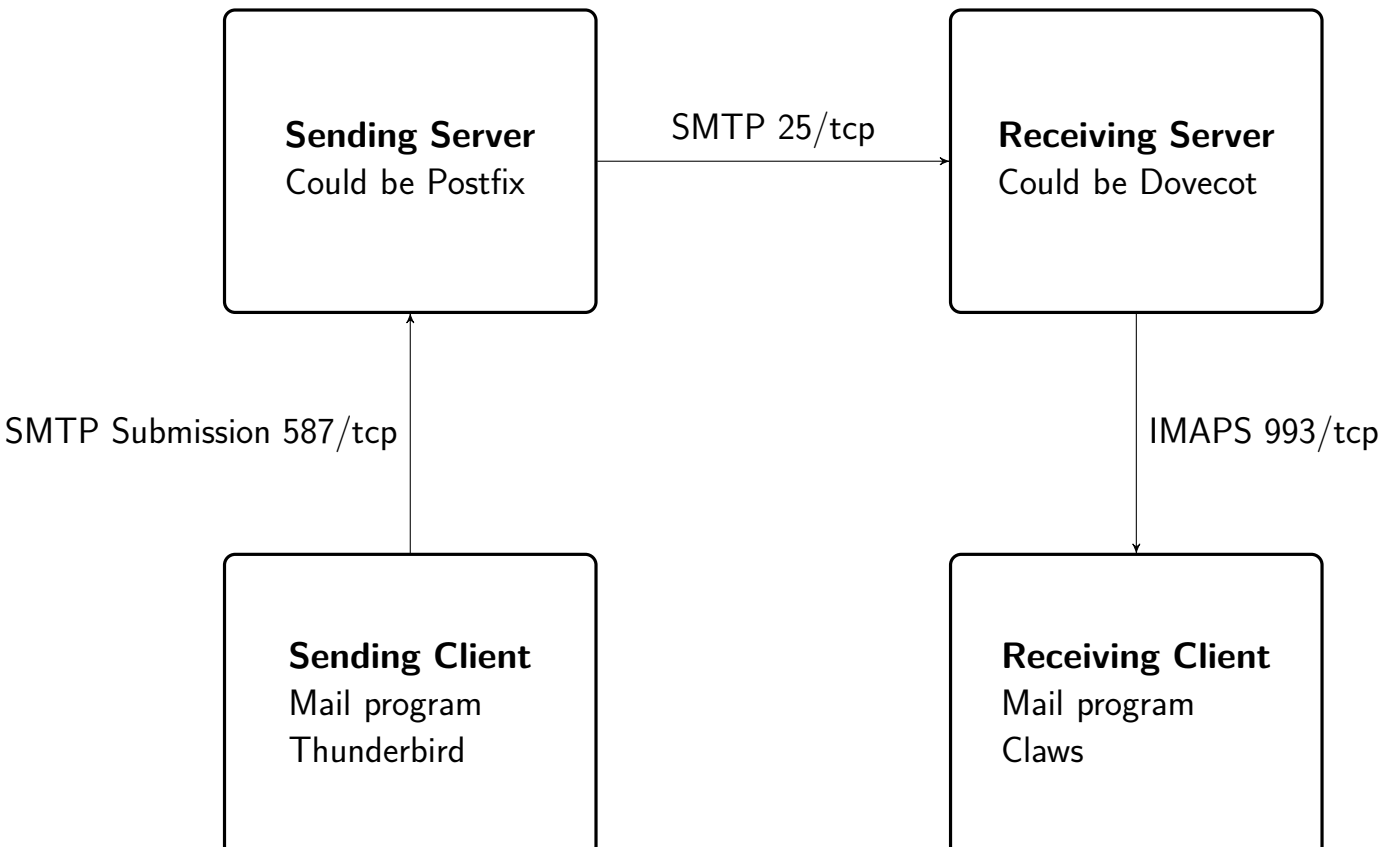
IMAP was designed with the goal of permitting complete management of an email box by multiple email clients, therefore clients generally leave messages on the server until the user explicitly deletes them. An IMAP server typically listens on port number 143. IMAP over SSL (IMAPS) is assigned the port number 993.

Virtually all modern e-mail clients and servers support IMAP, which along with the earlier POP3 (Post Office Protocol) are the two most prevalent standard protocols for email retrieval.[2] Many webmail service providers such as Gmail, Outlook.com and Yahoo! Mail also provide support for both IMAP and POP3.

https://en.wikipedia.org/wiki/Internet_Message_Access_Protocol

Recommendation: Only use the TLS enabled version on port 993/tcp

# Example of SMTP and IMAP



# Email Software



My recommendations are:

- Sending mail with SMTP using Postfix
- Receiving mail with IMAP using Dovecot

I would NOT use these:

- Exim - multiple Remote Code Execution over the years, fatal security vulns https://www.exim.org/static/doc/security/
- OpenSMTPD has had some really strange vulnerabilities recently https://www.opensmtpd.org/security.html
- Dont use Sendmail - old and horrible
- Other commercial email systems, evaluate for yourself

Many more exist: https://en.wikipedia.org/wiki/Comparison_of_mail_servers

# Postfix






Postfix is a free and open-source mail transfer agent (MTA) that routes and delivers electronic mail.

It is released under the IBM Public License 1.0 which is a free software license. Alternatively, starting with version 3.2.5, it is available under the Eclipse Public License 2.0 at the user's option.[2]

Originally written in 1997 by Wietse Venema at the IBM Thomas J. Watson Research Center in New York, and first released in December 1998,[3] Postfix continues as of 2024 to be actively developed by its creator and other contributors.

Source: https://en.wikipedia.org/wiki/Postfix_(software)

Home page: http://www.postfix.org/

# Dovecot



> Dovecot is an open-source IMAP and POP3 server for Unix-like operating systems, written primarily with security in mind.[3] Timo Sirainen originated Dovecot and first released it in July 2002. Dovecot developers primarily aim to produce a lightweight, fast and easy-to-set-up open-source email server.
>
> The primary purpose of Dovecot is to act as mail storage server. Mail is delivered to the server using some mail delivery agent (MDA) and stored for later access with an email client (mail user agent, or MUA).

Source: https://en.wikipedia.org/wiki/Dovecot_(software)

> Dovecot is an open source IMAP and POP3 email server for Linux/UNIX-like systems, written with security primarily in mind. Dovecot is an excellent choice for both small and large installations. It's fast, simple to set up, requires no special administration and it uses very little memory.

Home page: https://www.dovecot.org/

# Email security – mandatory



- SPF Sender Policy Framework

  https://en.wikipedia.org/wiki/Sender_Policy_Framework
- DKIM DomainKeys Identified Mail

  https://en.wikipedia.org/wiki/DomainKeys_Identified_Mail
- DMARC Domain-based Message Authentication, Reporting and Conformance

  https://en.wikipedia.org/wiki/DMARC
- Use them all

A huge part of email security is ensuring our domains are not abused in spoofing attacks, and spam

# Sender Policy Framework (SPF)



```
$ host -t TXT zencurity.com
zencurity.com descriptive text "v=spf1 a mx mx:kramse.dk -all"
```

Sender Policy Framework (SPF) is an email authentication method designed to detect forging sender addresses during the delivery of the email.[1] SPF alone, though, is limited only to detect a forged sender claimed in the envelope of the email which is used when the mail gets bounced.[1] **Only in combination with DMARC can it be used to detect the forging of the visible sender in emails** (email spoofing[2]), a technique often used in phishing and email spam.

SPF allows the receiving mail server to check during mail delivery that a mail claiming to come from a specific domain is submitted by an IP address authorized by that domain's administrators.[3] The list of authorized sending hosts and IP addresses for a domain is published in the DNS records for that domain.

Source:

https://en.wikipedia.org/wiki/Sender_Policy_Framework

# DomainKeys Identified Mail (DKIM)



DomainKeys Identified Mail (DKIM) allows the receiver to check that an email claimed to have come from a specific domain was indeed authorized by the owner of that domain.[1] It achieves this by affixing a digital signature, linked to a domain name, to each outgoing email message. The recipient system can verify this by looking up the sender's public key published in the DNS. A valid signature also guarantees that some parts of the email (possibly including attachments) have not been modified since the signature was affixed.[2] Usually, DKIM signatures are not visible to end-users, and are affixed or verified by the infrastructure rather than the message's authors and recipients.

Source:

https://en.wikipedia.org/wiki/DomainKeys_Identified_Mail

# Domain-based Message Authentication (DMARC)



DMARC (Domain-based Message Authentication, Reporting and Conformance) is an email authentication protocol. It is designed to give email domain owners the ability to protect their domain from unauthorized use, commonly known as email spoofing. The purpose and primary outcome of implementing DMARC is to protect a domain from being used in business email compromise attacks, phishing emails, email scams and other cyber threat activities.

Once the DMARC DNS entry is published, any receiving email server can authenticate the incoming email based on the instructions published by the domain owner within the DNS entry. If the email passes the authentication it will be delivered and can be trusted. If the email fails the check, depending on the instructions held within the DMARC record the email could be delivered, quarantined or rejected.

DMARC extends two existing mechanisms, Sender Policy Framework (SPF) and DomainKeys Identified Mail (DKIM).

DMARC Domain-based Message Authentication, Reporting and Conformance

Source:

https://en.wikipedia.org/wiki/DMARC

DMARC for non-sending Domains

If you have domains that *never send email* then add the following SPF and DMARC to avoid misuse.

```
v=spf1 -all
v=DMARC1; p=reject;
```

# You Don't Have to Do It Alone



Self-hosting does not mean doing everything yourself from scratch. There is a thriving ecosystem of communities, cooperatives, and associations that run shared infrastructure for their members – combining the benefits of self-hosting with shared operational responsibility.

- Associations and cooperatives can pool resources – shared hardware, shared skills, shared on-call burden
- Members retain control and trust in the people running the service, rather than a faceless corporation
- A great middle ground between full DIY and surrendering to Big Tech
- The following examples show real organisations doing exactly this – each with different scope, values, and technical approaches
- They may also inspire you to start something similar in your own community, workplace, or interest group



## Fripost - The Free Email Association

Technology has never been neutral. The ownership structure of technology companies and their endeavour for profit ensures that usage of their services for work and communication will never be free.

At the same time, only a fraction of the everyday Internet users have knowledge and resources enough to create free alternatives on their own. The question is also: What can reasonably be expected from an average user in terms of engagement in their privacy and freedom?

For any one individual, the goal of user freedom may be unattainable. Together, we can move mountains.

We are proud to present the Association for Free Email, Fripost. Fripost has since late 2010 delivered a free and reliable email service, with a distributed network of servers.

Source: https://fripost.org

# Some Examples: data.coop



## Velkommen til data.coop

data.coop er et kooperativ, som ejer og driver en digital infrastruktur for medlemmerne. Vores grundlæggende formål er at passe på medlemmernes data.

Vores kerneprincipper er:

Privatlivsbeskyttelse

- Vi er fælles om at beskytte vores data. Vi deler dem ikke for profit. Dine data transmitteres krypteret på nettet. Kryptering
- Vi tilbyder løsninger, der er sikre og grundigt deklarerede. Decentralisering
- Vores services snakker gerne sammen med andre decentrale services på nettet. Zero-knowledge
- Når det er muligt, sørger vi for, at systemadministratorer rent teknisk ikke kan tilgå medlemmernes data.

Ud fra de kerneprincipper vil vi med tiden udbyde onlinetjenester til medlemmerne. Hovedtanken er, at vi som udgangspunkt stoler mere på hinanden end på "de store" som f.eks. Google, Microsoft eller Facebook.

Source: https://data.coop

# Some Examples: kantree – buy from better companies



At Kantree, we believe in collaboration, transparency and trust. As a workers cooperative (100% employee-owned company), these values are deeply rooted. They are reflected in our products and in the relationship we build with each customer.

...

While a lot of **competitors are focusing on AI, we believe human interactions are the actual missing piece** to making digital collaboration tools better suited for the future of work. We should not strive to eliminate the human element from our tools, but rather take advantage of our collective intelligence.

... More autonomy implies a deeper involvement. It requires transparency to ensure a clear vision of the situation and of the objectives. It also requires processes which are adapted to each teams and which can evolve through time.

**These believes combined with our values led us to create Digicoop, the company behind Kantree.**

Source: https://kantree.io/about

# Things We Didn't Cover: File Sync and Collaboration



Replacing cloud storage and office collaboration tools is very achievable with self-hosted open source software

- **Nextcloud** – the most complete self-hosted collaboration platform: files, calendar, contacts, video calls, office documents, and more https://nextcloud.com
- **Syncthing** – decentralised file synchronisation between your own devices, no server required https://syncthing.net
- **Seafile** – fast, reliable file sync with a focus on performance and simplicity https://seafile.com
- **OnlyOffice** and **Collabora Online** integrate with Nextcloud to provide browser-based editing of Word, Excel, and PowerPoint compatible documents
- **Cryptpad** – end-to-end encrypted collaborative documents, spreadsheets, and kanban boards https://cryptpad.fr
- **Home Assistant** runs locally and keeps your smart home data off vendor clouds https://www.home-assistant.io
- The self-hosting community maintains **Awesome-Selfhosted** – a curated list https://awesome-selfhosted.net
- Many of these run well in containers – a `docker-compose.yml` or YAML and an NGINX reverse proxy is usually all you need to get started

# Things We Didn't Cover: Communication and Messaging



- **Mattermost** – open source Slack alternative, self-hosted, with threads, channels, and integrations https://mattermost.com
- **Matrix / Element** – federated, encrypted messaging and team chat; run your own homeserver with Synapse or Conduit https://matrix.org
- **Jitsi Meet** – self-hosted video conferencing, no account required, works in the browser https://jitsi.org
- **Rocket.Chat** – team messaging with video, file sharing, and a large plugin ecosystem https://rocket.chat
- **XMPP** – the original open federated messaging protocol, still widely used, clients include Conversations (Android) and Gajim (desktop)
- Federation matters: Matrix and XMPP allow communication across different servers – you are not locked into a single provider or instance

# Things We Didn't Cover: Monitoring, Passwords, and More



- **Monitoring** – **LibreNMS** for network monitoring https://librenms.org, **Grafana** + **Prometheus** for metrics and dashboards https://grafana.com
- **Logging** – **VictoriaLogs** for central log storage and search, single binary, works with Grafana https://docs.victoriametrics.com/victorialogs/
- **Password management** – **Bitwarden** offers an official self-hosted server https://bitwarden.com; **Vaultwarden** is a lightweight community reimplementation in Rust, compatible with all Bitwarden clients and easier to run on modest hardware
- **MFA** – hardware keys like **YubiKey** https://www.yubico.com and **Nitrokey** https://www.nitrokey.com https://github.com/dani-garcia/vaultwarden
- **DNS filtering** – **Pi-hole** block ads and tracking domains network-wide via DNS https://pi-hole.net
- **Git hosting** – **Gitea** or **Forgejo** provide a lightweight self-hosted GitHub alternative for code and documentation https://forgejo.org

# Things We Didn't Cover: Client Operating Systems



- **Debian** – rock-solid, well-supported Linux desktop and laptop OS; what we use at Zencurity for workstations https://www.debian.org
- **Ubuntu** – Debian-based, more beginner-friendly, large community and good hardware support; a natural first step from Windows or macOS https://ubuntu.com
- **Fedora** – Red Hat sponsored, cutting-edge packages, good for developers https://fedoraproject.org
- **Linux Mint** – often recommended as the most approachable desktop Linux for people switching from Windows https://linuxmint.com
- **Qubes OS** – security through compartmentalisation; runs each application or context in a separate VM (qube); not for everyone, but very powerful for high-security workstations – used at Zencurity https://www.qubes-os.org
- All of the above are free, open source, and run well on modest hardware – including older laptops that Windows 11 refuses to support

# Conclusion: Take Control, Take Responsibility



Self-hosting is not for everyone – but for those who choose it, it offers genuine independence, real learning, and ownership of your data

- **Start small** – a single VPS with NGINX and UFW is already self-hosting
- **Use open standards** – OCI images, open protocols, open formats keep your options open
- **Version control everything** – if it is not in Git, it does not exist
- **Security is not optional** – firewalls, TLS, patching, and backups from day one
- **You are on call** – know this going in, and plan for it
- **Communities help** – cooperatives, associations, and friendly sysadmins share the burden and the knowledge
- **Digital sovereignty is a journey** – every service you move away from Big Tech is a step toward independence

# Questions?



Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

You are always welcome to send me questions later via email

Mobile: +45 2026 6000

Email: hlk@zencurity.com